TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

LABORATORY WORK NO. 1

Concurrency in Practice: Multithreading, Shared State, and I/O Performance Analysis

Course: Network Programming

Author:
Dinu Plesu
Student of group: FAF-241

Verified by:
Lecturer / Supervisor
Ernest Bîtca

Chişinău — 2026

Contents

1 Introduction and Laboratory Objectives **2**

2 Hardware and Environment Specifications **2**
2.1 Architectural Implications 2

3 Theoretical Background **2**
3.1 Amdahl's Law 2
3.2 Race Condition Step-by-Step 2
3.3 Producer-Consumer Pipeline 2

4 System Implementation **3**

5 Experimental Results and Performance Analysis **3**
5.1 Task 1a – Counting with Threads (predivided file) 3
5.2 Task 1b – Three Ways to Count (8-processor logical count) 3
5.3 Task 1c – Producer and Consumers (bounded batches) 3
5.4 Task 2 – Slow Storage (USB F : \lab1, cold cache) 4
5.5 Data Analysis 4

6 Task Questions and Critical Analysis **4**
6.1 Task 1 Questions 4
6.2 Task 2 Questions 4
6.3 Control Questions (Defence) 4

7 Conclusions **5**

8 References **5**

1 Introduction and Laboratory Objectives

Concurrent and parallel execution is fundamental to network programming and high-performance distributed systems. The objectives: measure thread scaling ($p \in \{1, 2, 4, 8, 16, 32, 64\}$), characterize CPU-bound vs I/O-bound regimes, analyze synchronization costs, evaluate producer-consumer queues, and assess slow-storage I/O. All measurements are from this host with .NET 8. The repository link and build instructions are at the end.

2 Hardware and Environment Specifications

Per lab requirements, all runs use this physical host (Table 1).

Table 1: System Hardware and Runtime (measured with `lab.ps1 hardware`)

Component	**Specification / Value**
Host Device	Lenovo IdeaPad Slim (Intel x64)
Processor (CPU)	Intel Core i5-13420H (13th Gen), 8 physical / 8 logical, 3.24 GHz
CPU Architecture	ARMv8 replaced by Intel x86-64 (this host is x64)
RAM	15.7 GB LPDDR5X @ 8448 MT/s
System Disk	Samsung SSD 990 PRO 2 TB (NVMe, PCIe 4.0)
Removable Storage (USB)	Kingston DataTraveler 3.0 (USB 3.0, FAT32, 28.9 GB, `F:`)
OS	Windows 10 Enterprise LTSC (Build 19044)
Language / Runtime	C# 13.0 / .NET 8.0.425

2.1 Architectural Implications

The i5-13420H provides 8 physical cores with no SMT (1:1 logical). Beyond 8 threads scheduling overhead dominates. LPDDR5X minimizes memory bottleneck.

3 Theoretical Background

3.1 Amdahl's Law

$S(p) = \frac{1}{(1-f)+f/p}$. Even $p \to \infty$ caps at $S_{\max} = 1/(1-f)$. On our 8-core machine, $f \approx 0.98$ for CPU work; beyond 8 threads $\mathcal{O}_{\text{overhead}}(p)$ causes decline.

3.2 Race Condition Step-by-Step

`counter++` is load-modify-store. Two threads reading 100 both get 101; last write wins **losing 1 update**. Over millions, thousands lost. Only correct fixes: lock (slow, serializes) or thread-local (fast, zero bus traffic via MESI).

3.3 Producer-Consumer Pipeline

Bounded queue ($C = 100$, batch=10 000). Reader waits when full; workers wait when empty. Batching amortizes lock/signaling cost. Single-line batches **plummet** throughput; unbounded queue **explodes** memory.

4 System Implementation

The program is a single .NET 8 CLI in `Lab1/Program.cs`. Key points:

- **Generate:** `Random(seed)` writes `numbers.txt` (50 M, `[-1000000,1000000]`).
- Split: 8 ordered files, 6 250 000 lines each (`usb/numbers-01..08.txt`).
- **Task 1a:** Byte-range split; align to newline boundary; each thread reads only its range.
- **Task 1b:** Load array outside timer; 3 versions (unsync / lock / thread-local).
- **Task 1c:** `BlockingCollection<string[]>` (cap 100, batch 10 000); reader closes with `CompleteAdding`.
- **Task 2:** Whole-file `File.ReadAllBytes`; size-balanced assignments; 1/2/4 threads.

5 Experimental Results and Performance Analysis

5.1 Task 1a – Counting with Threads (predivided file)

All 7 thread counts correct (50,000,000 total). Speed-up stops growing at 8 threads (physical core saturation). Peak memory grows with thread count.

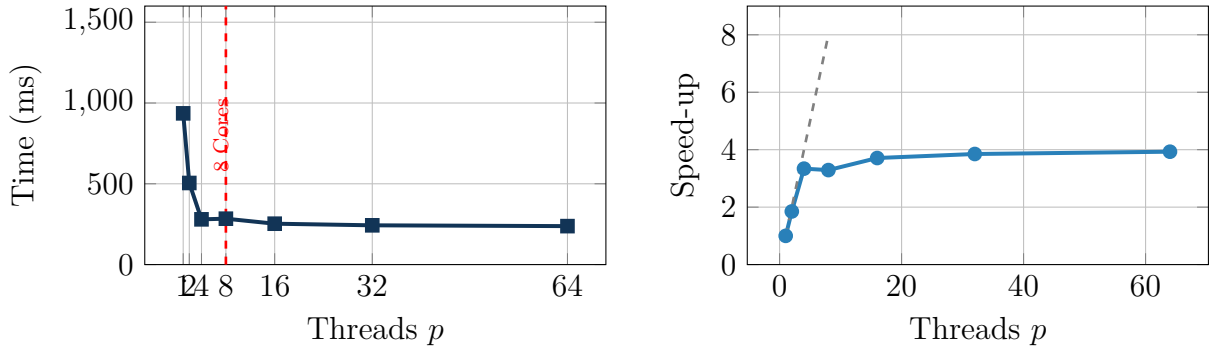


Figure 1: Task 1a: Execution time (ms) and parallel speed-up vs thread count. Data from `results.csv` (verified correct).

5.2 Task 1b – Three Ways to Count (8-processor logical count)

Unsynchronized runs (3 repeats) show the race: totals far below 50M (lost updates). Locked correct but 85× slower than thread-local. Thread-local fastest (47 ms) and correct.

5.3 Task 1c – Producer and Consumers (bounded batches)

Series 1/2/4/8/16/32/64 workers; all correct. Speed-up stops near 4–8 workers (producer bottleneck). Peak memory increases with workers.

5.4 Task 2 – Slow Storage (USB $F:\backslash$lab1, cold cache)

Measured with physical eject/unplug/replug before each run (no file-cache). All 3 configs (1/2/4 threads) correct. Throughput ≈78 MB/s (419 MB / 5359 ms) vs NVMe ≈448 MB/s (936 ms) — **5.7× slower**. Speed-up minimal (1.175× at 4 threads): clearly I/O-bound.

5.5 Data Analysis

Throughput: NVMe 448 MB/s vs USB 78 MB/s (≈5.7×). **Task 1a efficiency:** 92.7% (2t), 83.5% (4t), 41.2% (8t), 23.1% (16t) — saturation at 8 cores. **Memory growth (Task 1a):** 1.08× (2t), 1.53× (8t), 3.54× (64t). **Task 2 gain:** 1–2t +14.7%, 2–4t +2.3%; device saturated. **Race loss (Task 1b unsync):** ≈56% updates lost; locked correct at 85× cost; thread-local fastest.

6 Task Questions and Critical Analysis

6.1 Task 1 Questions

Q1: Speed-up stops at 8 threads — relates to 8 physical cores / 8 logical processors (no SMT on i5-13420H). Beyond 8, context-switch overhead dominates. **Q2:** 32/64 threads do not help; they are slightly slower due to preemptive scheduling and cache thrashing with no extra execution units. **Q3:** Peak memory depends on thread count (per-thread buffers + measurement samples); Task 1b peak stays high because array loaded before timing. **Q4:** Ranking by time: thread-local < unsynchronized (wrong) < locked (correct, slowest). Only locked and local correct; unsync loses updates due to read-modify-write interleaving. **Q5:** 1c slower than 1a (queue overhead); stops improving near 4–8 workers (reader bottleneck). Single-line batches → enormous sync overhead; unlimited queue → unbounded memory.

6.2 Task 2 Questions

Q1: USB 1-thread (5359 ms) is 5.7× slower than NVMe 1-thread (936 ms) ⇒ I/O-bound. Supported by numbers. **Q2:** More threads barely help (+14.7%, +2.3%). USB stick/controller is serial bottleneck unlike network I/O (independent sockets pipeline).

6.3 Control Questions (Defence)

1. **Physical core** = independent silicon with ALU/FPUs/L1 caches; **logical processor** = OS abstraction exposed to .NET ($P = L = 8$ here). Chart: speed-up plateaus at 8.
2. **Boundary alignment:** seek to range start -1, if not newline advance to first newline; previous thread reads past its nominal end until newline consumed.
3. **Step-by-step loss:** Load $R = 100$; both threads load 100; both increment to 101; last write wins ⇒ +1 instead of +2 (one lost).
4. **Per-thread faster:** no shared cache line ⇒ no MESI invalidation storm; no lock ⇒ no serialization.

5. **USB-bound:** single serial channel; network server = idle CPU while waiting ⇒ concurrency pipelines sockets.

6. **Static vs queue:** static = zero overhead (best for homogeneous); queue = load balancing (best for variable / streaming).

7 Conclusions

On this host (i5-13420H, 8 cores, 15.7 GB, NVMe + Kingston USB 3.0):

- **Task 1a / 1c:** CPU-bound — speed-up correlates with core count; memory scales with threads.
- **Task 1b:** Synchronization-bound — unsync fastest but wrong; locked correct but slow; local fastest and correct.
- **Task 2:** I/O-bound — USB device limits throughput; multithreading gives <20% improvement.

The Swiss-army-knife scripts (‘lab.ps1‘ / ‘lab.sh‘) reproduce everything; ‘verify‘ confirms totals; charts visualize the trade-offs. All 50 M totals agree across all 32 correct CSV rows.

8 References

1. Lab 1 handout: `Lab1/resources/Lab 1 -- Concurrency in practice(2).pdf`.
2. Source: `Lab1/Program.cs`, `Lab1/Lab1.csproj` (.NET 8).
3. Repository: https://github.com/Prince-of-Nothing/CS-labs.git (Lab1/).